

```

// Project UID 1d9f47bfc76643019cfbf037641defe1

#include "Player.h"
#include "Card.h"
#include "unit_test_framework.h"

#include <iostream>

using namespace std;

//all cards so can call them easier
Card nine_spades = Card(Card::RANK_NINE, Card::SUIT_SPADES);
Card ten_spades = Card(Card::RANK_TEN, Card::SUIT_SPADES);
Card jack_spades = Card(Card::RANK_JACK, Card::SUIT_SPADES);
Card queen_spades = Card(Card::RANK_QUEEN, Card::SUIT_SPADES);
Card king_spades = Card(Card::RANK_KING, Card::SUIT_SPADES);
Card ace_spades = Card(Card::RANK_ACE, Card::SUIT_SPADES);

Card nine_clubs = Card(Card::RANK_NINE, Card::SUIT_CLUBS);
Card ten_clubs = Card(Card::RANK_TEN, Card::SUIT_CLUBS);
Card jack_clubs = Card(Card::RANK_JACK, Card::SUIT_CLUBS);
Card queen_clubs = Card(Card::RANK_QUEEN, Card::SUIT_CLUBS);
Card king_clubs = Card(Card::RANK_KING, Card::SUIT_CLUBS);
Card ace_clubs = Card(Card::RANK_ACE, Card::SUIT_CLUBS);

Card nine_hearts = Card(Card::RANK_NINE, Card::SUIT_HEARTS);
Card ten_hearts = Card(Card::RANK_TEN, Card::SUIT_HEARTS);
Card jack_hearts = Card(Card::RANK_JACK, Card::SUIT_HEARTS);
Card queen_hearts = Card(Card::RANK_QUEEN, Card::SUIT_HEARTS);
Card king_hearts = Card(Card::RANK_KING, Card::SUIT_HEARTS);
Card ace_hearts = Card(Card::RANK_ACE, Card::SUIT_HEARTS);

Card nine_diamonds = Card(Card::RANK_NINE, Card::SUIT_DIAMONDS);
Card ten_diamonds = Card(Card::RANK_TEN, Card::SUIT_DIAMONDS);
Card jack_diamonds = Card(Card::RANK_JACK, Card::SUIT_DIAMONDS);
Card queen_diamonds = Card(Card::RANK_QUEEN, Card::SUIT_DIAMONDS);
Card king_diamonds = Card(Card::RANK_KING, Card::SUIT_DIAMONDS);
Card ace_diamonds = Card(Card::RANK_ACE, Card::SUIT_DIAMONDS);

Card two_spades = Card(Card::RANK_TWO, Card::SUIT_SPADES);
Card two_diamonds = Card(Card::RANK_TWO, Card::SUIT_DIAMONDS);

TEST(test_player_get_name) {
    Player * alice = Player_factory("Alice", "Simple");
    ASSERT_EQUAL("Alice", alice->get_name());

    delete alice;
}

TEST(test_make_trump_simple)
{
    Player * playa = Player_factory("playaa", "Simple");

    playa->add_card(queen_spades);
    playa->add_card(jack_spades);
    playa->add_card(jack_clubs);
    playa->add_card(ten_clubs);
    playa->add_card(ace_hearts);
}

```

```

    //two or more face cards of faceup, round 1
    string trump1;
    ASSERT_TRUE(playa->make_trump(nine_spades, true, 1, trump1));
    ASSERT_TRUE(playa->make_trump(nine_spades, false, 1, trump1));

    delete playa;
}

TEST(test_make_trump_extra)
{
    Player * playa = Player_factory("playaa", "Simple");

    playa->add_card(queen_spades);
    playa->add_card(jack_spades);
    playa->add_card(jack_clubs);
    playa->add_card(ten_clubs);
    playa->add_card(ace_hearts);

    //one face card of faceup
    string trump1;
    ASSERT_TRUE(playa->make_trump(nine_clubs, true, 1, trump1));
    ASSERT_TRUE(playa->make_trump(nine_clubs, false, 1, trump1));
    ASSERT_EQUAL(trump1, Card::SUIT_CLUBS);

    //no face card of faceup
    ASSERT_FALSE(playa->make_trump(nine_diamonds, true, 1, trump1));
    ASSERT_FALSE(playa->make_trump(nine_diamonds, false, 1, trump1));
    ASSERT_FALSE(playa->make_trump(nine_hearts, true, 1, trump1));
    ASSERT_FALSE(playa->make_trump(nine_diamonds, false, 1, trump1));
    ASSERT_EQUAL(trump1, Card::SUIT_CLUBS);

    delete playa;
}

TEST(test_add_and_discard)
{
    Player * playa = Player_factory("playa", "Simple");
    playa->add_card(nine_clubs);
    playa->add_card(nine_clubs);
    playa->add_card(nine_clubs);
    playa->add_card(nine_clubs);
    playa->add_card(nine_clubs);
    playa->add_and_discard(queen_clubs);
    Card test = playa->lead_card(Card::SUIT_HEARTS);
    ASSERT_EQUAL(test, queen_clubs);

    delete playa;
}

TEST(test_add_and_discard_bower)
{
    Player * playa = Player_factory("playa", "Simple");
    playa->add_card(queen_spades);
    playa->add_card(queen_spades);
    playa->add_card(queen_spades);
    playa->add_card(king_spades);
    playa->add_card(queen_spades);
    playa->add_and_discard(jack_spades);

```

```

    Card test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_spades);

    delete playa;
}

TEST(test_lead_card)
{
    Player * playa = Player_factory("playa", "Simple");
    playa->add_card(nine_spades);
    playa->add_card(ten_spades);
    playa->add_card(jack_spades);
    playa->add_card(queen_spades);
    playa->add_card(king_spades);
    //spades is trump
    Card test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_spades);

    delete playa;
}

TEST(test_lead_card_2)
{
    Player * playa = Player_factory("playa", "Simple");
    playa->add_card(nine_spades);
    playa->add_card(ten_spades);
    playa->add_card(jack_spades);
    playa->add_card(queen_spades);
    playa->add_card(jack_clubs);
    //spades is trump
    Card test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_spades);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_clubs);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, queen_spades);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, ten_spades);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, nine_spades);

    delete playa;
}

TEST(test_lead_card_no_trump)
{
    Player * playa = Player_factory("playa", "Simple");
    playa->add_card(nine_hearts);
    playa->add_card(ten_hearts);
    playa->add_card(jack_hearts);
    playa->add_card(queen_hearts);
    playa->add_card(ace_hearts);
    //spades is trump
    Card test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, ace_hearts);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, queen_hearts);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_hearts);
}

```

```

    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, ten_hearts);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, nine_hearts);

    delete playa;
}

TEST(test_lead_card_3) //this one wrong
{
    Player * playa = Player_factory("playa", "Simple");
    playa->add_card(queen_hearts);
    playa->add_card(nine_clubs);
    playa->add_card(queen_spades);
    playa->add_card(jack_spades);
    playa->add_card(two_diamonds);
    //spades is trump
    //should be: queen hearts, nine clubs, two diamonds, jack spades, queen spades
    Card test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, queen_hearts);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, nine_clubs);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, two_diamonds);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_spades);
    test = playa->lead_card(Card::SUIT_SPADES);
    ASSERT_EQUAL(test, queen_spades);

    delete playa;
}

TEST(test_euchre_00)
{
    Player * a = Player_factory("a", "Simple");
    a->add_card(jack_hearts);
    a->add_card(nine_diamonds);
    Card test = a->play_card(jack_spades, Card::SUIT_HEARTS);
    ASSERT_EQUAL(test, nine_diamonds);
}

TEST(play_card)
{
    Player * a = Player_factory("a", "Simple");
    a->add_card(queen_spades);
    a->add_card(jack_spades);
    a->add_card(jack_clubs);
    a->add_card(ten_clubs);
    a->add_card(ace_hearts);
    Card nine_spades = Card(Card::RANK_NINE, Card::SUIT_SPADES);

    Card test = a->play_card(nine_spades, Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_spades);
    test = a->play_card(nine_spades, Card::SUIT_SPADES);
    ASSERT_EQUAL(test, jack_clubs);
}

```

```

test = a->play_card(nine_spades, Card:: SUI_T_SPADES);
ASSERT_EQUAL(test, queen_spades);
test = a->play_card(nine_spades, Card:: SUI_T_SPADES);
ASSERT_EQUAL(test, ten_clubs);
test = a->play_card(nine_spades, Card:: SUI_T_SPADES);
ASSERT_EQUAL(test, ace_hearts);

a->add_card(jack_diamonds);
a->add_card(jack_hearts);
a->add_card(ten_spades);
a->add_card(queen_clubs);
a->add_card(ace_hearts);
//trump diamonds
test = a->play_card(ace_clubs, Card:: SUI_T_DIAMONDS);
ASSERT_EQUAL(test, queen_clubs);
test = a->play_card(queen_clubs, Card:: SUI_T_DIAMONDS);
ASSERT_EQUAL(test, ten_spades);
test = a->play_card(nine_hearts, Card:: SUI_T_DIAMONDS);
ASSERT_EQUAL(test, ace_hearts);
test = a->play_card(ten_hearts, Card:: SUI_T_DIAMONDS);
ASSERT_EQUAL(test, jack_hearts);
test = a->play_card(ten_hearts, Card:: SUI_T_DIAMONDS);
ASSERT_EQUAL(test, jack_diamonds);

a->add_card(jack_clubs);
a->add_card(ten_clubs);
a->add_card(ace_spades);
a->add_card(nine_hearts);
a->add_card(two_spades);

test = a->play_card(ace_clubs, Card::SUI_T_SPADES);
ASSERT_EQUAL(test, ten_clubs);
test = a->play_card(queen_spades, Card::SUI_T_SPADES);
ASSERT_EQUAL(test, jack_clubs);
test = a->play_card(ten_hearts, Card::SUI_T_SPADES);
ASSERT_EQUAL(test, nine_hearts);
test = a->play_card(ace_clubs, Card::SUI_T_SPADES);
ASSERT_EQUAL(test, two_spades);
test = a->play_card(ten_hearts, Card::SUI_T_SPADES);
ASSERT_EQUAL(test, ace_spades);

```

```

}

```

```

TEST(test_play_hand_edge)

```

```

{

```

```

    Player * a = Player_factory("a", "Simple");
    a->add_card(ace_diamonds);
    a->add_card(king_diamonds);
    a->add_card(nine_clubs);
    a->add_card(ten_clubs);
    a->add_card(queen_diamonds);
    Card test = a->lead_card(Card::SUI_T_SPADES);
    ASSERT_EQUAL(test, ace_diamonds);
    test = a->lead_card(Card::SUI_T_SPADES);
    ASSERT_EQUAL(test, king_diamonds);
    test = a->play_card(queen_spades, Card::SUI_T_SPADES);
    ASSERT_EQUAL(test, nine_clubs);

```

```

    test = a->play_card(jack_hearts, Card::SUIT_SPADES);
    ASSERT_EQUAL(test,ten_clubs);
    test = a->play_card(ten_diamonds, Card::SUIT_SPADES);
    ASSERT_EQUAL(test,queen_diamonds);
}

TEST(test_play_edge_2)
{
    Player * a = Player_factory("a", "Simple");
    a->add_card(nine_diamonds);
    a->add_card(queen_clubs);
    a->add_card(jack_clubs);
    a->add_card(king_spades);
    a->add_card(ten_spades);
    Card test = a->play_card(ace_clubs, Card::SUIT_DIAMONDS);
    ASSERT_EQUAL(test,queen_clubs);
    test = a->play_card(nine_clubs,Card::SUIT_DIAMONDS);
    ASSERT_EQUAL(test,jack_clubs);
    test = a->play_card(queen_spades,Card::SUIT_DIAMONDS);
    ASSERT_EQUAL(test,king_spades);
    test = a->play_card(jack_hearts,Card::SUIT_DIAMONDS);
    ASSERT_EQUAL(test,nine_diamonds);
    test = a->play_card(ten_diamonds,Card::SUIT_DIAMONDS);
    ASSERT_EQUAL(test,ten_spades);
}

TEST(add_card_basic)
{
    Player * a = Player_factory("a", "Simple");
    a->add_card(ace_spades);
    Card test = a->play_card(ace_clubs, Card::SUIT_DIAMONDS);
    ASSERT_EQUAL(test,ace_spades);
}

TEST(test_print_operator)
{
    Player * a = Player_factory("Abraham Lincoln", "Simple");
    ostream ostring;
    ostring << *a;
    ostream correct;
    correct << "Abraham Lincoln";
    ASSERT_EQUAL(ostring.str(),correct.str());
}

TEST(test_9_clubs_Chi_Chih)
{
    Player * a = Player_factory("Chi-CHih", "Simple");
    a->add_card(queen_spades);
    a->add_card(nine_clubs);
    a->add_card(ten_clubs);
    Card test = a->play_card(ten_diamonds,Card::SUIT_HEARTS);
    ASSERT_EQUAL(test,nine_clubs);
}

```

```
// Add more tests here  
TEST_MAIN()
```